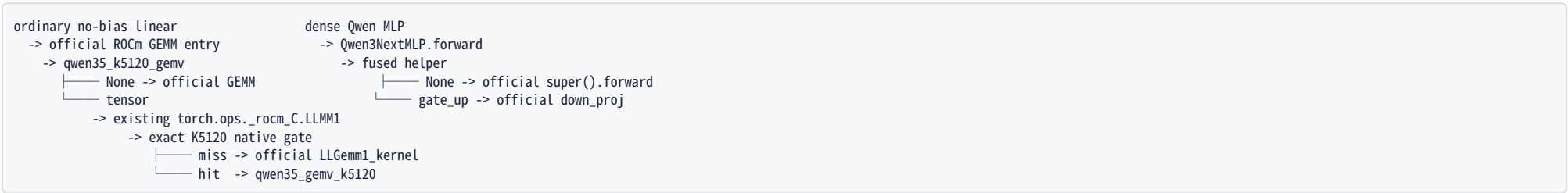


Owner C 新手闭卷指南：K5120 GEMV 与 GateUp/SwiGLU

目标：不熟悉 HIP/C++ 也能依靠官方 vLLM 的现有 LLMM1 ABI，重新实现固定 K=5120 fast path、Python 窄 gate 和 GateUp/SwiGLU 融合。本文不修改源码，也不在非目标设备构建 或实验。

0. 一页总图



闭卷只背：

不建新 extension；Python 先试、失败回官方；LLMM1 内 early return、失败留原 dispatch。

1. C 负责的四层

层	职责	失败怎么办
设备门	缓存 <code>is_gfx936()</code>	返回 <code>false</code>
Python shape gate	只接 K5120 目标 BF16 shape	返回 <code>None</code>
模型/linear 接入	在官方入口前尝试 helper	继续官方实现

层	职责	失败怎么办
_rocm_C.LLMM1 native	固定 K kernel 和 fused SwiGLU	继续官方 dispatch

C 是 gfx936.py 文件 owner，但 A 的 GDN 函数段由 A 负责语义。公共设备判断还影响 A 的 attention/GDN，并间接影响 B page784。

2. 官方积木：少背代码的搜索地图

```
# 官方 LLMM1、LLGemm1 kernel、类型 dispatch、stream、输出分配
rg -n "LLGemm1_kernel|torch::Tensor LLMM1|AT_DISPATCH" \
  csrc/rocm/skinny_gemms.cu

# 官方 Python ROCm GEMM 分派
rg -n "def rocm_unquantized_gemm_impl" \
  vllm/model_executor/layers/utils.py

# 官方 MLP gate_up -> activation -> down_proj
rg -n "class Qwen2MoeMLP|def forward|gate_up_proj|down_proj" \
  vllm/model_executor/models/qwen2_moe.py

# 官方 ROCm 设备属性读取
rg -n "get_device_properties|gcnArchName|RocmPlatform" vllm/platforms/rocm.py

# extension 在 setup.py 的接入方式
rg -n "_rocm_C|CMakeExtension|ext_modules" setup.py CMakeLists.txt
```

vLLM v0.18.1 官方入口：

- https://github.com/vllm-project/vllm/blob/v0.18.1/csrc/rocm/skinny_gemms.cu
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/model_executor/layers/utils.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/model_executor/models/qwen2_moe.py
- <https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/platforms/rocm.py>
- <https://github.com/vllm-project/vllm/blob/v0.18.1/setup.py>

FIND IN OFFICIAL	OWNER C MUST DERIVE	
LLMM1 ABI and binding	exact K/M/dtype/layout gate	
output allocation + stream	5120/8=640 thread mapping	
BF16 conversion idioms	two-level reduction	
official fallback dispatch	2/4 rows-per-CTA choice	
Qwen2MoeMLP forward order	BF16-staged fused GateUp semantics	

4. Python 窄 gate

4.1 支持条件

```
K == 5120
M in {96, 14336, 16384, 34816, 248320}
x.numel() == 5120
x and weight are BF16
weight contiguous
x.stride(-1) == 1
x on GPU and device is gfx936
fuse_silu only when M == 34816
ordinary linear additionally requires bias is None
```

不满足返回 None，不能抛错抢走官方通用路径。

4.2 为什么用 `x.numel()==5120`

这保证总共只有一个输入向量，而不仅是最后一维等于 5120。目标是 decode GEMV，不是多 token GEMM。输出 reshape 回 `(*x.shape[:-1], M)`，fused 时最后一维为 `M/2`。

4.3 公共设备门

设备判断只回答“是不是 gfx936”，不混入 shape、dtype 或模型逻辑，并用 cache 避免重复查询：

```
device id -> get_device_properties -> gcnArchName startswith gfx936:
    |
    | cached bool shared by A/C gates
```

CPU 路径不能盲目调用 CUDA device properties；调用者必须先确认张量在 CUDA/ROCm device。

5. 不新增 ABI：复用 `_rocm_C.LLMM1`

Python 继续调用：

```
torch.ops._rocm_C.LLMM1(weight, x.reshape(1,5120), rows_per_block)
```

特殊的负 `rows_per_block=-2` 仅作为现有调用点内部的 fused 标记；公共 operator 签名不变。

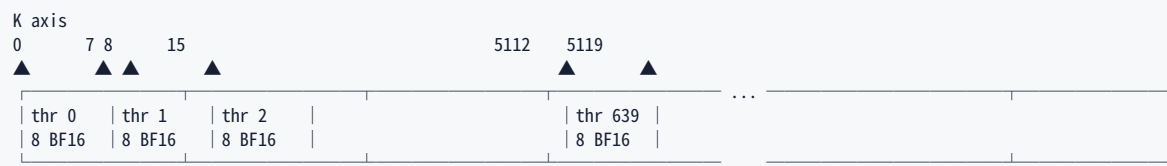
LLMM1 内的安全结构：

```
official TORCH_CHECKs
  |
  ▼
recognize exact K5120 BF16 shapes
  |—— hit -> launch specialized kernel -> early return
  |—— miss -> untouched official AT_DISPATCH + LLGemm1_kernel
```

不要创建 `_rocm_C2`、新 pybind 名称或另一套 extension。这样可以复用官方 build、binding、stream、device guard 和 fallback。

6. HIP kernel: 640 个 thread 怎样合作

6.1 第一层：每线程 8 个元素



`dot_bfloat16x8` 将 8 个 weight 和 8 个 input 转成四组 BF16 pair，使用 FP32 FMA 累加，得到每个 `(row, thread)` 的局部 sum。

6.2 为什么是 wave64 和 10 waves

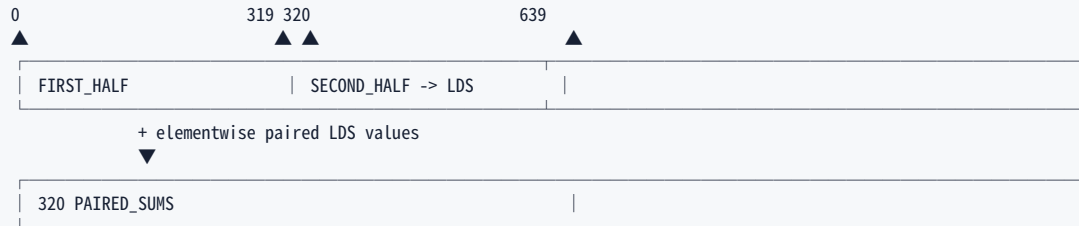
gfx936 的 wavefront size 为 64；640 threads 形成 10 waves：

```
thread 0..319 = first half = 5 waves
thread 320..639 = second half = 5 waves
```

6.3 第二层前的 paired halves

后 320 threads 把局部 sum 写入 LDS `halves[row][0..319]`；同步后，前 320 threads 将对应位置加到自己 sum，于是 640 个 partial sum 变为 320 个。

Tensor: partial sums per output row, length=640
 Formula: $\text{paired}[i] = \text{sum}[i] + \text{sum}[i+320]$

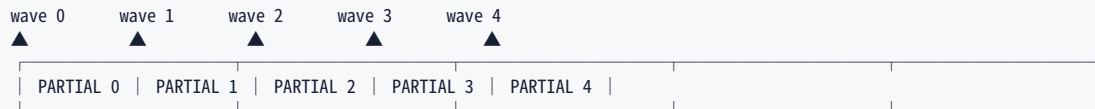


`__syncthreads()` 必须位于所有线程都能到达的位置；不能只让分支内部分线程同步。

6.4 wave 内 shuffle

前 320 threads 是 5 个 waves。每个 wave 用 XOR shuffle 将 64 个值规约为 1 个，wave leader 写 `reductions[row][wave]`。

Tensor: wave reductions, shape=[ROWS,5]
 Formula: $\text{wave_sum}[w] = \text{sum}(\text{paired}[w*64:(w+1)*64])$



6.5 最终规约

同步后最前面的 ROWS 个 threads 各自读取 5 个 wave sum，得到一个 output row 的 FP32 total，最后转 BF16 存储。

```
y[row] = reduction[row][0] + ... + reduction[row][4]
```

6.6 ROWS=2 和 ROWS=4

- M=96：每 CTA 处理 4 行，`grid=96/4`。
- 其他普通 M：每 CTA 处理 2 行，`grid=M/2`。

kernel 模板参数 ROWS 让编译器展开每个 thread 对多个 output row 的工作。

7. GateUp/SwiGLU 融合

7.1 官方数学顺序

官方 MLP:

```
gate_up = linear(x, W_gate_up)    # [34816] = [gate 17408 | up 17408]
hidden  = SiLU(gate) * up         # [17408]
output  = down_proj(hidden)       # [5120]
```

C 只融合前两步，down projection 仍调用官方层。

7.2 两遍 launch

Tensor: W_gate_up, shape=[34816,5120]
Formula: first half produces gate; second half produces up

M row axis



第一次使用普通 kernel 将 gate 写进最终 output buffer。第二次使用 FUSE_SILU=true：算 up，读取 buffer 中 BF16 gate，先按官方 staging 将 SiLU 结果舍入 BF16，再与 BF16 up 相乘并写回。

7.3 为什么强调 BF16-staged

如果一直用 FP32 gate/up 到最终乘法，数学更“精确”，但可能不再逐项匹配官方 linear BF16 output -> SiluAndMul 的舍入路径。闭卷实现目标是与官方语义对齐，不是擅自改变精度路径。

7.4 模型 fallback

Qwen3NextMLP 继承官方 Qwen2MoeMLP，只覆盖 forward：

```
expert_gate is None and helper succeeds
-> official self.down_proj(fused_hidden)
otherwise
-> super().forward(x)
```

不复制 `__init__`，因此 weight loader、TP sharding 和其他官方行为自然保留。

8. 五个 M 从哪里来

只支持有 Qwen3.5 调用证据的输出宽度：

	M	来源类别
	96	GDN 小投影
	14336	GDN/模型目标投影
	16384	GDN 目标投影
	34816	dense MLP GateUp
	248320	LM head

现场不要凭记忆扩展 shape；从 Qwen 构造函数中的 projection weight shape 重新推导。额外 M 会扩大 correctness surface，却没有目标 layer 调用证据。

9. 构建：C 与 A/B 最大的区别

C 修改 `csrc/rocm/skinny_gemms.cu`，属于 native HIP 源码。Python 文件更新不等于 `.so` 更新：


```

skinny_gemms.cu + existing binding/CMake
    ↓
wheel native build
    ↓
vllm/_rocm_C*.so in wheel
    ↓
isolated install + Python helpers
    ↓
torch.ops._rocm_C.LLMM1 uses new kernel

```

只设置 `PYTHONPATH=` 可能得到“新 Python + 旧 `_rocm_C.so`”，不能验证 C。原 Owner C 文档 要求恢复官方 `setup.py` 已预留的 `vllm._rocm_C` extension，并构建完整 wheel。

当前设备不对，因此本指南不执行构建或实验。目标比赛设备上执行时必须打印实际加载的 `.so` 路径，并确认来自新隔离安装目录。

目标比赛容器中的完整流程如下；这里只记录，不代表本文编写环境执行过：

```

REPO_ROOT=/public/home/tangyu408/Qwen_DCU_Worker_0/pr2026-bh408-gqa-page784-k5120
BUILD_SOURCE=/tmp/qwen35-build-source
ARTIFACT_ROOT=/tmp/qwen35-build
SITE_DIR="$ARTIFACT_ROOT/site"

cd "$REPO_ROOT"
git worktree add --detach "$BUILD_SOURCE" HEAD
mkdir -p "$ARTIFACT_ROOT"/{dist,bdist,cache}
cd "$BUILD_SOURCE"
VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
  build --build-base "$ARTIFACT_ROOT/build" \
  bdist_wheel --bdist-dir "$ARTIFACT_ROOT/bdist" \
  --dist-dir "$ARTIFACT_ROOT/dist"
python3 -m pip install --no-deps --target "$SITE_DIR" \
  "$ARTIFACT_ROOT/dist/vllm-*.whl

cd "$REPO_ROOT"
OFFICIAL=fa718036bdb9dfd80a872b86c8ac16c9d02bfd31
git clang-format --diff "$OFFICIAL" -- csrc/rocm/skinny_gemms.cu
RUN_DIR="$(mktemp -d /tmp/qwen35-k5120-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 PYTHONPATH="$SITE_DIR" \
  python3 "$REPO_ROOT/docs/csrc/verify_qwen35_optimizations.py" k5120 swiglu)

```

构建前必须确认 `setup.py` 的 ROCm 分支确实把 `vllm._rocm_C` 加入 `ext_modules`；安装后用 模块/动态库路径 确认新 `.so`，而不是仅凭 Python helper 的路径判断。若临时 `worktree` 已存在，先检查来源，不要盲目覆盖或删除。

10. 闭卷实现顺序

10.1 Native

1. 打开官方 `skinny_gemms.cu`，找到 LLMM1 与 LLGemm1。
2. 保留 ABI、TORCH_CHECK、device guard、stream 和官方 dispatch。
3. 写 BF16x8→FP32 dot helper。
4. 写 `template<ROWS,FUSE_SILU> kernel`。
5. 640 threads 各覆盖 K 的 8 个元素。
6. 后 320 写 LDS，前 320 paired add。
7. 5 个 wave leader 写第二级 LDS。
8. 前 ROWS threads 合并 5 个数并写 BF16。
9. fused 时读取第一遍 gate，按 BF16 staging 算 SiLU×up。
10. LLMM1 只对精确 shape early return，其余落回官方 dispatch。

10.2 Python

1. 写缓存的纯设备判断。
2. 写 K/M/dtype/layout/numel/gfx936 gate，失败返回 None。
3. 普通 linear 只在 `bias is None` 时尝试。
4. dense MLP 只在 `expert_gate is None` 时尝试 fused。
5. fused 成功后调用已有 `down_proj`；失败 `super().forward`。

11. 十条闭卷不变量

1. 复用 `_rocm_C.LLMM1`，不创建新 ABI/extension。
2. Python unsupported 返回 None，调用者继续官方路径。
3. native unsupported 落回原 `AT_DISPATCH`。
4. $K=5120$ ， $BF16 \times 8$ ，所以正好 640 threads。
5. $640=2 \times 320=10$ waves；paired 后只剩 5 waves。
6. 所有 partial sum 和规约使用 FP32。

7. M=96 用 4 rows/CTA，其余普通目标 M 用 2。
8. fused 仅 M=34816，输出宽度减半为 17408。
9. fused 顺序与 BF16 staging 必须匹配官方 GateUp→SiluAndMul。
10. down_proj、expert path、bias path 和非目标设备保留官方实现。

12. 从官方 LLMM1 重建的最小伪代码

这些骨架刻意省略 HIP 类型名和精确 API。闭卷时先恢复结构，再从同文件官方 kernel 复制编译 正确的类型转换、stream 和 launch 语法。

12.1 Python helper

```
def qwen35_k5120_gemv(weight, x, fuse_silu=False):
    M, K = weight.shape
    supported = (
        K == 5120
        and M in TARGET_M
        and (not fuse_silu or M == 34816)
        and x.numel() == 5120
        and x.dtype == weight.dtype == bfloat16
        and weight.is_contiguous()
        and x.stride(-1) == 1
        and x.is_cuda
        and is_gfx936(x.device)
    )
    if not supported:
        return None

    mode = FUSED_SENTINEL if fuse_silu else (4 if M == 96 else 2)
    out = torch.ops._rocm.C.LLMM1(weight, x.reshape(1, 5120), mode)
    return out.reshape(*x.shape[:-1], M // 2 if fuse_silu else M)
```

12.2 Native LLMM1 插入点

```
Tensor LLMM1(weight, input, rows_per_block) {
    // Keep official shape/dtype TORCH_CHECKs first.
    M = weight.size(0); K = weight.size(1);
    fused = rows_per_block == FUSED_SENTINEL && M == 34816;
    target = K == 5120 && input_is_bf16 && M_in_target_set;
    output = allocate_on_same_device({1, fused ? M / 2 : M});
    stream = official_current_stream();

    if (target) {
        if (fused) {
            launch<2, false>(grid=M/4, threads=640,
                             first_half_weight, input, null_gate, output);
            launch<2, true>(grid=M/4, threads=640,
                           second_half_weight, input, output_as_gate, output);
        } else if (M == 96) {
            launch<4, false>(grid=M/4, threads=640, ...);
        } else {
            launch<2, false>(grid=M/2, threads=640, ...);
        }
        return output;
    }

    // Paste/preserve the complete official AT_DISPATCH fallback here.
}
```

fused 必须同时受 sentinel 和 M=34816 保护；普通目标 M 不能因负值模式错误分配 M/2。

12.3 HIP kernel 骨架

```
template<int ROWS, bool FUSE_SILU=false>
kernel(weight, input, gate, output) {
    thread = threadIdx.x;          // 0..639
    lane = thread % 64;            // 0..63
    wave = thread / 64;            // 0..9
    row_start = blockIdx.x * ROWS;

    x8 = vector_load_8_bf16(input, thread);
    for row in 0..ROWS-1:
        w8[row] = vector_load_8_bf16(weight, row_start+row, thread)
        sum[row] = dot_bf16x8_to_fp32(w8[row], x8)

    if thread >= 320:
        halves[row][thread-320] = sum[row]
    sync_all_threads()

    if thread < 320:
        sum[row] += halves[row][thread]
        sum[row] = wave64_shuffle_reduce(sum[row])
        if lane == 0:
            reductions[row][wave] = sum[row] // wave is now 0..4
    sync_all_threads()

    if thread < ROWS:
        total = sum(reductions[thread][0..4])
        if FUSE_SILU:
            gate_bf16 = gate[row_start+thread]
            up_bf16 = round_to_bf16(total)
            total = round_to_bf16(silu(gate_bf16)) * up_bf16
        output[row_start+thread] = round_to_bf16(total)
}
```

伪代码中第二个 LDS 索引的 `wave` 只有前 320 threads 会写，因此范围是 0..4。若直接让后五个 wave 写入 `[ROWS][5]`，就会越界。

12.4 两个 Python 接入点

```
def official_rocm_gemm(x, weight, bias=None):
    if bias is None:
        result = qwen35_k5120_gemv(weight, x)
        if result is not None:
            return result
    # Continue the untouched official implementation.

class Qwen3NextMLP(Qwen2MoeMLP):
    def forward(self, x):
        if self.expert_gate is None:
            hidden = qwen35_k5120_gemv(self.gate_up_proj.weight, x, True)
            if hidden is not None:
                return self.down_proj(hidden)[0]
        return super().forward(x)
```

13. 静态审查与目标设备验收

非目标设备只做源码/ABI/公式审查。真正设备验收依据原 Owner C 文档：

- 检查专用代码位于 LLMM1 TORCH_CHECK 后、官方 dispatch 前；
- 检查所有 `__syncthreads()` 可被整个 block 到达；
- 手算 640 threads 对 K=5120 无重无漏；
- 手算 grid 对 M=96 和其他 M 无越界；
- 普通 M 五种输出与 `F.Linear` 对齐；
- fused 输出与 `SiluAndMul(F.Linear(...))` 对齐且 shape=17408；
- FP16、非连续 weight、多向量 x、unsupported M、bias、expert gate 全部 fallback；
- CPU/非 gfx936 设备门为 false；
- 打印并确认新 `_rocm_C.abi3.so` 路径；
- 最终同一 wheel 做 DP1/DP2 服务冒烟。

14. 最常见错误

- 新 Python 生效但结果没变：加载的是旧 `_rocm_C.so`。
- 多 token 输入误入：只检查 `x.shape[-1]`，忘了 `x.numel()==5120`。

- 非目标模型出错：helper 失败时抛异常而不是返回 None。
- 某些 M 尾部错：grid/ROWS 不能整除或越界未处理。
- 结果约一半：只规约了 320 threads，漏加后半。
- wave 规约错：把 NVIDIA warp32 习惯套到 AMD wave64。
- fused 数值不一致：没有保留 BF16 gate/up staging。
- expert 模型语义错：expert_gate 非 None 仍走 fused dense path。
- bias 丢失：通用 linear 有 bias 时仍提前返回 fast result。
- 构建破坏通用形状：删除/改动了 LLMM1 官方 dispatch fallback。

15. 口述答案

C 在 Python 中用缓存的 gfx936 判断和精确 K/M/BF16/layout gate，只为单个 K5120 向量调用已有 `_rocm_C.LLMM1`；unsupported 返回 None，让通用 linear 或官方 MLP 继续。LLMM1 ABI 不变，在官方检查后识别目标 shape 并 early return，其他输入仍走原 LLGemm1。专用 HIP kernel 使用 640 threads，每线程读取 8 个 BF16，得到 FP32 partial sum；后 320 threads 经 LDS 与前 320 配对，前五个 wave64 各自 shuffle 规约，再由前 ROWS 个线程合并五个 wave sum。M=96 每 CTA 四行，其余两行。GateUp 对 34816 行做两遍，第一遍把 BF16 gate 暂存输出，第二遍算 BF16 up 并按官方 staging 写 $\text{SiLU}(\text{gate}) \times \text{up}$ 的 17408 维结果，down_proj 仍用官方层。因为这是 native HIP 修改，必须完整构建新 wheel 并确认加载新 `_rocm_C.so`，不能用新 Python 配旧二进制。

16. 十轮自审记录

以下十轮以 Owner C 原说明和当前 Python/HIP 源码为依据；只做静态审查，未运行 native 构建 或设备实验。

轮次	检查主题	检查结果与完善
1	职责覆盖	设备门、Python gate、linear/MLP 接入、LLMM1 native 全覆盖
2	官方模板	补齐 skinny GEMM、GEMM dispatch、Qwen2MoeMLP、ROCm、setup 搜索图
3	ABI/接口	明确不建新 extension、None fallback、native early return
4	shape/grid	复算 K5120、五个 M、普通 2/4 rows、fused M/2
5	kernel 数学	复算 BF16x8×640、FP32 dot、paired halves、五 wave sum

轮次	检查主题	检查结果与完善
6	同步/内存	核对 LDS 两级规约、wave64、全 block 可达的同步边界
7	SwiGLU语义	核对 gate/up 两遍、BF16 staging、17408 输出与官方 down_proj
8	fallback/gate	覆盖 bias、expert、FP16、非连续、多向量、非 gfx936、其他 shape
9	构建/装载	补齐完整 wheel 流程，强调新 Python/旧 .so 风险和路径确认
10	可视化/新手性	检查 K/M 轴、thread/wave 区域、平直边框、十条不变量与口述答案

最终静态结论：本文可用于从官方 LLMM1 骨架重建目标实现；native 正确性、allclose 和性能仍 必须在原 Owner C 指定的 gfx936/ROCm 比赛环境中验证，当前审查不声称替代设备验收。